

SOFTWARE ENGINEERING LAB-3

Name: Upadrasta Naga Sarvani

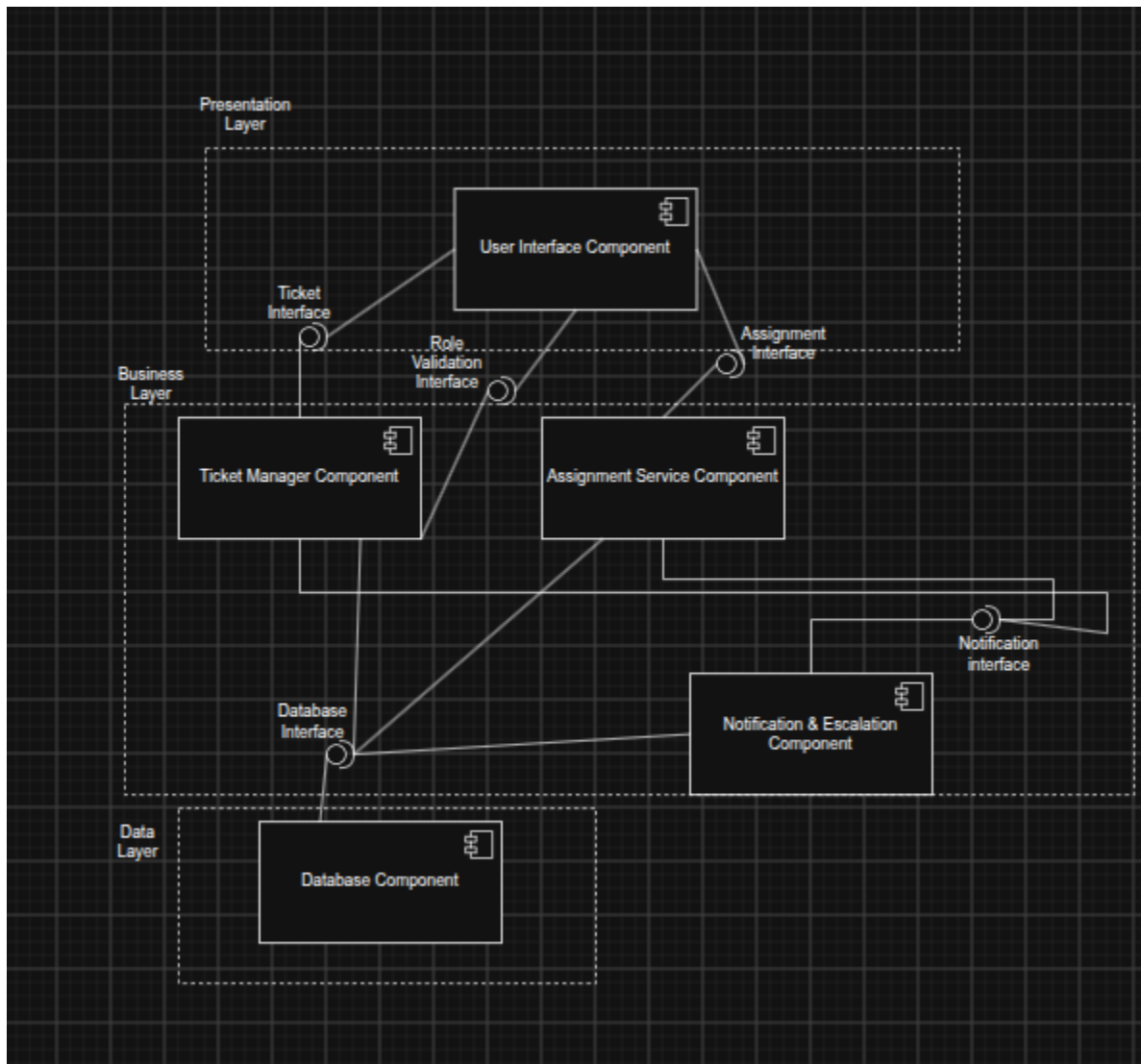
SRN: PES2UG24CS567

Sem-5, Section J

Scenario: Hostel Maintenance & Issue Ticketing System.

Github link: <https://github.com/sarvaniun/Hostel-Maintenance-Ticketing-System>

Component Diagram:



Architectural Justification

We chose Layered Architecture for the Hostel Maintenance & Issue Ticketing System.

Architectural Choice

The system is organized into three layers: a Presentation Layer (User Interface Component), a Business Layer (Ticket Manager, Assignment Service, and Notification & Escalation Components), and a Data Layer (Database Component). Each layer communicates only with the layer directly beside it through well-defined interfaces.

Reason 1: Clear Mapping to Sequential Business Rules

The system's core workflow is inherently sequential: a resident logs a ticket, a warden reviews and assigns it, staff update its status, and the resident confirms closure (UC-01 through UC-05). This step-by-step flow maps naturally onto layered validation and processing — the Presentation Layer captures input, the Business Layer enforces rules such as validation (UC-06) and role restriction (NFR-002), and the Data Layer persists the result. A layered structure keeps each stage of this workflow isolated and easy to trace.

Reason 2: Appropriate Scale and Simplicity

The system serves a single hostel with three actor roles and one cohesive ticketing workflow, not multiple independently-scaling services. Microservices would introduce unnecessary operational complexity (network calls, distributed data consistency) for a system of this scope, while Client-Server alone is too coarse-grained to show the internal component structure the system needs. Layered Architecture gives the right level of separation without over-engineering the design.

Security Advantage

Role-based access control (NFR-002) is enforced centrally in the Business Layer through the Role Validation Interface, rather than being scattered across individual screens. Because the Presentation Layer never talks to the Database Component directly, every request — whether from a resident, warden, or maintenance staff member — must pass through business-layer checks first. This prevents a user from bypassing role restrictions by manipulating the UI layer directly, and keeps a single, auditable point of enforcement.

Performance Benefit

Isolating the Notification & Escalation Component within the Business Layer allows SLA escalation checks (NFR-001) to run independently of the main ticket-handling path. A resident logging a new ticket or a warden reviewing the queue does not wait on escalation processing, since it communicates through its own Notification Interface rather than blocking the primary request-response flow between the Presentation and Data layers.